

React.js

What Do Web Developers Use Today?

- Depending on the company and software stack used, almost every website will be paired with a JavaScript library to perform more difficult tasks
- [jQuery](#)
- [Angular.js](#)
- [Vue.js](#)
- [React.js](#)

What Exactly is React?

- React is a JavaScript library for building robust user interfaces
- Declarative:
 - Design simple views for each state of your application and React will efficiently update and render the right components when your data changes
- Component-based:
 - Build components that manage their own state
 - Component logic is written in JavaScript instead of templates
- You can incorporate React into any existing web project
 - Simple to add a React component to a web page
 - [React Native](#)
 - [Apps built using React Native](#)

Advantages

- Virtual DOM: Much more efficient way to update the view
 - Write pure JavaScript that updates React components and React updates DOM for you
- Easy readability and maintainability
- Render React on the server side
- Can use React with any framework (Backbone.js, Angular.js) since it is only the view layer

What is Needed for React?

- All that is needed is Node.js installed ([windows](#) [mac](#))
 - We need npm (see next slides)
- We will then create React apps using the [vite](#) build tool which is managed by the team that build vue.js
 - It will help automatically generate files and directory structure

Once Node.js is Installed - Node Version

- The command “node -v” lists the current version of Node.js that is installed on your machine.
- Make sure you don't have anything below v14

```
jirani@csmbirani ~ % node -v  
v18.6.0
```

Node.js - npm

- Npm stands for node package manager
- It comes built in when Node.js is downloaded so no need to worry about doing any extra work
- It is a package manager that helps with package installation and different node modules
 - We will cover all of this later in the course, for now just know it helps manage different Node.js libraries to help/assist with project development

Node.js - npm Version

- Use the command “npm -v” to see what version of Node Package Manager is installed on your machine

```
jals-mbp:~ jalirani$ npm -v  
5.3.0  
jals-mbp:~ jalirani$ █
```

Node.js - npm List

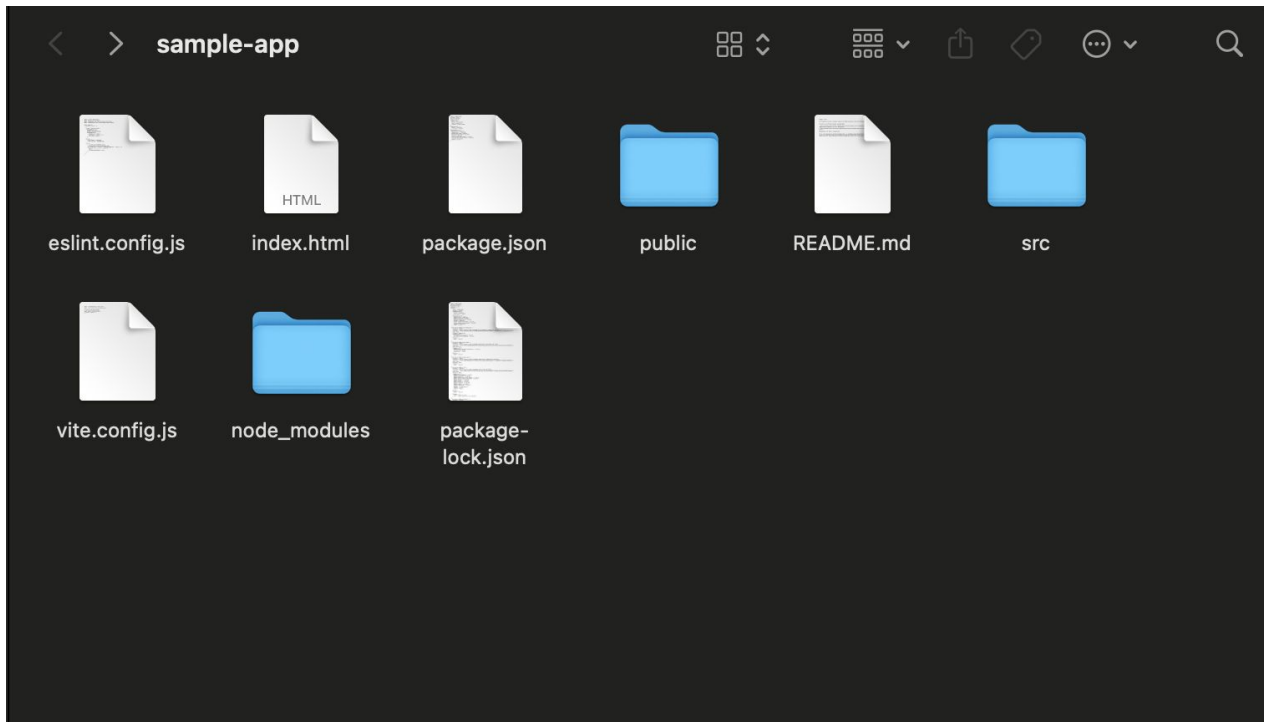
- Use the command “npm ls” to see all of the packages that are installed on your machine

```
jals-mbp:~ jalirani$ npm ls
/Users/jalirani
├─┬ bl@1.2.1
│   └─┬ readable-stream@2.3.3
│       ├── core-util-is@1.0.2
│       ├── inherits@2.0.3
│       ├── isarray@1.0.0
│       ├── process-nextick-args@1.0.7
│       ├── safe-buffer@5.1.1 deduped
│       └─┬ string_decoder@1.0.3
│           └─┬ safe-buffer@5.1.1 deduped
│               └─┬ util-deprecate@1.0.2
└─┬ cheerio@1.0.0-rc.2
```

Getting Started - Tutorial

- First let's use the vite wizard to set up the project
 - Type "npm create vite@latest"
 - Project name: sample-app, Select a framework: React, language: "JavaScript"
- Now "cd sample-app"
- Install Vite in the project "npm install -D vite" (you can do this globally)
- "npm run dev" to run the project
 - This will generate a sample-app folder with a few directories and files

Create-react-app Scaffolding



Node Modules Directory

- I would completely ignore this directory, it never needs to be touched
- The purpose is it has all node packages imported into the project
 - Babel will automatically bring in a bunch
 - When submitting your projects, delete this folder because it is massive and will take forever to upload

Package.json Files

- Package-lock.json: describes the tree from module installs. No need to worry or touch this
- Package.json: Shows the startup scripts that are run when certain commands are typed “start”, “build”, etc. We will dive into them shortly
 - The dependencies key is the actual React actually being imported
 - In a lot of cases, React is imported in the script tags at the top of an HTML file
 - Facebook abstracted these dependencies

Public Directory

- Manifest.json: provides metadata used when your webapp is added to the homescreen on Android.
- Robots.txt: website letting web crawlers know what part of the website should or should not be scanned
- Index.html: Is the root HTML file that is loaded. React components are loaded inside of this HTML file.
 - You typically only want to make static changes such as <title> etc.

Src

- App.css: is the style sheet for the App.js file, which is where the components live (JSX code, I will explain this soon)
- Index.css: is the style sheet for the root index.html file
- Index.js: Loads the App component and places it in the “root” id (which was present in the index.html) via the ReactDOM

Src (cont.)

- `App.test.js`: Is a spec file used to write unit tests. It is initially loaded with a test to see if the page loads
- `App.js`: Is where your React Components are stored. Initially loaded with the logo and a few simple HTML tags (JSX).

Run Your App

- From the terminal, run “npm run dev” which will automatically host your website on a port
- Once an app is running, no need to stop, make changes, and rerun
 - React allows you to make changes while the site is live!
 - Once you save your changes the browser will automatically refresh

Building Your App

- Meta has a cool feature where you can release “builds” of your current web application
- Run “npm run build” and this will automatically generate a build folder to be put on a webserver

Building Your App (cont.)

- Having separate .css and .js files all over the places is good for development (smaller pieces to code), but can be slow in production due to so many HTTP requests
- This merges all .css files into one, and all .js into one
 - Much more necessary in high-traffic sites to save users a few milliseconds

Serve

- Once the build folder has been created, you can serve the build
 - Aka deploying with a static server
- “npm install serve”
 - Then “serve -s build”
- The difference between doing this and npm start is that this “build” is shared across your network for people to view

Before Development, Setup Your Environment!

- Any text editor will suffice, but Babel Syntax highlighting is incredibly important
- VS Code is the best option
- You can use Sublime Text 3 which is convenient for JavaScript development
 - Syntax highlighting can be an issue
 - [Sublime Babel Package](#)

JSX

- A statically-typed, object-oriented programming language designed to run on modern browsers
- JSX is fast because it performs optimizations before compiling down to JavaScript
- Very easy adaptability
- Can look like a template language, but works very well to create a UI in React

JSX Compiler

- If you ever want to verify jsx code, there is an npm package that will allow you to run via the command line
 - I don't really recommend doing this frequently since almost always there will be UI elements involved, which cannot be run via command line
 - [JSX package](#)
- Use “jsx --run filename.jsx” to run your code
 - Remember to embed you JSX code in a Main class and function (see documentation and example)

JSX Simple Example

- Below is an example of JSX
- At first glance it does seem a little strange setting a variable equal to “HTML”, but this is in fact, the essence of JSX

```
const classTitle = <h3>Welcome to Class!</h3>
```

Using a JSX Variable

- The `{...}` notation allows access to the value of a JSX variable

```
function App() {  
  const schoolName = 'Towson U';  
  const element = <h1>Welcome to, {schoolName}</h1>;  
  return (  
    <div className="App">  
      <header className="App-header">  
        {element}  
      </header>  
    </div>  
  );  
}
```

A Little Less Simple

- What displays?

```
function App() {
  const instructor = {
    fName: 'Jal',
    lName: 'Irani'
  };
  const formattedInstructorName = (
    <h1>
      Your instructor is: {formatName(instructor)}!
    </h1>
  );
  return (
    <div className="App">
      <header className="App-header">
        {formattedInstructorName}
      </header>
    </div>
  );
}

function formatName(user) {
  return user.fName + ' ' + user.lName;
}
```

Attributes with JSX

- Can also embed straight into attributes
 - Technically embedding JavaScript

```
const element = <img src={user.avatarUrl}></img>;
```

JSX is also an Expression

- After compilation, JSX expressions become regular JavaScript function calls and evaluate to regular JavaScript objects
- That means it is fair game to use inside of conditionals, for loops, accept it as arguments, return it, etc.

JSX Expressions

```
function getInstructor(user) {  
  // If user is not null  
  if (user) {  
    return <h1>Hello, {getName(user)}!</h1>;  
  }  
  return <h1>Hello, Unknown Person!</h1>;  
}
```

Another Important Note

- ReactDOM uses camelCase property naming conventions instead of HTML naming conventions
 - So “className” instead “class”
 - [ReactDOM Elements](#)

JSX at the Core

- Babel, the JSX compiler, compiles down to [React.createElement\(\)](#) calls

```
const element = (  
  <p className="test">  
    Good evening!  
  </p>  
);  
  
const element = React.createElement(  
  'p',  
  {className: 'test'},  
  'Good evening!'  
);
```

React Only Updates What's Necessary

- From the React website: “ReactDOM compares the element and its children to the previous one, and applies only the DOM updates necessary to bring the DOM to the desired state”
- No need to do a full refresh of the page
 - <https://youtu.be/lw2BLUjQo1E?t=11>
 - Nice example using paint flash

ReactDOM/VirtualDOM

Hello, world!

It is 12:26:46 PM.

```
Console Sources Network Timeline
▼<div id="root">
  ▼<div data-reactroot>
    <h1>Hello, world!</h1>
    ▼<h2>
      <!-- react-text: 4 -->
      "It is "
      <!-- /react-text -->
      <!-- react-text: 5 -->
      "12:26:46 PM"
      <!-- /react-text -->
      <!-- react-text: 6 -->
      "."
      <!-- /react-text -->
    </h2>
  </div>
</div>
```

Components

- Components let you split the UI into independent reusable pieces
- You can take smaller components and use them in a bigger component
 - Which is good design in general
- Similar to how in Java you want to abstract code into different methods and classes, Components allow a logical and reusable way

Function Components

- Returns a React Element, so it is valid

```
function ReactExample(props) {  
  ...  
  return <p>Welcome: {props.name}</p>;  
}
```

ES6 React Component Definition

- Equivalent to previous example
- render() function is where your JSX code will go
 - Typically will be what is displayed on the screen

```
class ReactExample extends React.Component {  
  render() {  
    return <p>Welcome: {this.props.name}</p>;  
  }  
}
```

Building Components

- As mentioned, good design is to build components within components
- It is key sometimes to sketch out your UI to see what parts can be made into abstractable components
- Reuse is key!

Before we Continue... Let's Talk About Props

- Which stands for “properties” is simply data passed to the component
- It is a JavaScript object that triggers a re-render when changed
- Props are read-only!
- See [W3 Example](#)

Rendering a Component

- `<componentName/>`
 - `Props.name?`
 - `name=?`

```
function ReactExample(props) {  
  return <p>Welcome: {props.name}</p>;  
}  
  
function App() {  
  return (  
    <div className="App">  
      <header className="App-header">  
        <ReactExample name="Jal Irani" />  
      </header>  
    </div>  
  );  
}
```

Props Breakdown

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
const element = <Welcome name="Sara" />;  
ReactDOM.render(  
  element,  
  document.getElementById('root')  
);
```

Breakdown

1. We call ReactDOM.render() with the `<Welcome name="Sara" />` element.
2. React calls the Welcome component with `{name: 'Sara'}` as the props.
3. Our Welcome component returns a `<h1>Hello, Sara</h1>` element as the result.
4. React DOM efficiently updates the DOM to match `<h1>Hello, Sara</h1>`.

Back to Our Regular Programming

Commonly Used Example of Extracting Components

- Custom Comment Component
- Contains many div elements
 - All are linked to classes
- There is an img tag
 - Properties, classes, etc

```
function Comment(props) {  
  return (  
    <div className="Comment">  
      <div className="UserInfo">  
        <img className="Avatar"  
          src={props.author.avatarUrl}  
          alt={props.author.name}  
        />  
        <div className="UserInfo-name">  
          {props.author.name}  
        </div>  
      </div>  
      <div className="Comment-text">  
        {props.text}  
      </div>  
      <div className="Comment-date">  
        {formatDate(props.date)}  
      </div>  
    </div>  
  );  
}
```

Avatar Component

```
function Avatar(props) {  
  return (  
    <img className="Avatar"  
      src={props.user.avatarUrl}  
      alt={props.user.name}  
    />  
  );  
}
```

```
function Comment(props) {  
  return (  
    <div className="Comment">  
      <div className="UserInfo">  
        <Avatar user={props.author} />  
        <div className="UserInfo-name">  
          {props.author.name}  
        </div>  
      </div>  
      <div className="Comment-text">  
        {props.text}  
      </div>  
      <div className="Comment-date">  
        {formatDate(props.date)}  
      </div>  
    </div>  
  );  
}
```

UserInfo Component

```
function UserInfo(props) {  
  return (  
    <div className="UserInfo">  
      <Avatar user={props.user} />  
      <div className="UserInfo-name">  
        {props.user.name}  
      </div>  
    </div>  
  );  
}
```

```
function Comment(props) {  
  return (  
    <div className="Comment">  
      <UserInfo user={props.author} />  
      <div className="Comment-text">  
        {props.text}  
      </div>  
      <div className="Comment-date">  
        {formatDate(props.date)}  
      </div>  
    </div>  
  );  
}
```